

Seguridad, Resiliencia y Contingencia DevOps

Implementación de DevSecOps, alta disponibilidad y planes de recuperación en un sistema de gestión de inventarios construido con Angular 22, Node.js + Express y SQLite. Despliegue automatizado con Docker, Ansible y PM2.

Angular 22 Node.js + Express SQLite Docker Ansible DevSecOps Husky

CKJ - Sistema de Almacén © 2026

1. Índice

La infraestructura DevOps del proyecto CKJ se estructura en tres pilares fundamentales:

Seguridad / DevSecOps

Implementado: JWT, Helmet, bcrypt, Rate Limit, validación, Husky pre-commit, lint-staged, Gitleaks, npm audit, Trivy, SonarQube, Snyk, Checkov, Fail2Ban, UFW, Hardening SSH

Propuesto: Terraform + AWS WAF, Kubernetes Secrets, RDS PostgreSQL

Resiliencia

Implementado: PM2 auto-reinicio + watch + 300MB RAM máx, Docker multi-stage + restart, Ansible deploy + rollback, Nginx proxy + SSL + caché

Propuesto: Kubernetes con 3 réplicas + HPA, AWS Auto Scaling, ALB

Contingencia

Implementado: saveDb() persistencia inmediata, Rate limiting anti-DoS, UFW firewall, Rollback Ansible, Hardening preventivo

Propuesto: RDS Multi-AZ, K8s PodDisruptionBudget, S3, DR multi-zona

2. Seguridad Implementada - Código (1/2)

Helmet + Rate Limiting CÓDIGO REAL

backend/server.js - Cabeceras HTTP seguras y límite de peticiones

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
app.use(helmet()); // XSS, clickjacking, MIME sniffing
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 100 // por IP
});
app.use('/api/', limiter);
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 5 // 5 intentos
});
app.post('/api/login', loginLimiter, ...);
```

¿Qué hace? Helmet añade cabeceras HTTP de seguridad (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection). Rate limiter evita ataques de fuerza bruta limitando a 100 peticiones por IP cada 15 minutos, y solo 5 intentos de login.

JWT + Middleware de Roles CÓDIGO REAL

backend/server.js - Autenticación y control de acceso basado en roles (RBAC)

```
function authMiddleware(rolesPermitidos = []) {
  return (req, res, next) => {
    const token = req.headers['authorization']?.split(' ')[1];
    if (!token) return res.status(401).json({error: 'Token'});
    const decoded = jwt.verify(token, JWT_SECRET);
    const user = queryOne('SELECT * FROM usuarios WHERE id=?', [decoded.id]);
    if (!user) return res.status(401).json({error: 'No válido'});
    if (rolesPermitidos.length && !rolesPermitidos.includes(user.rol))
      return res.status(403).json({error: 'Permisos'});
    req.user = user; next();
  };
}
// Cada ruta especifica roles permitidos:
app.get('/api/usuarios', authMiddleware(['admin']), ...);
app.post('/materiales', authMiddleware(['admin', 'almacen']), ...);
```

¿Qué hace? Cada ruta declara qué roles pueden acceder. El middleware verifica el token JWT (expira en 12h), busca el usuario en BD, y comprueba su rol. Si no tiene permisos, responde con 403 Forbidden.

3. Seguridad Implementada - Código (2/2)



bcrypt - Hash de Contraseñas CÓDIGO REAL

backend/server.js - Contraseñas protegidas con 10 rondas de sal

```
const bcrypt = require('bcryptjs');
// Al crear usuario: hash con 10 rondas
const hashed = bcrypt.hashSync(password, 10);
db.run('INSERT INTO usuarios VALUES (NULL,?,?,?,1)',
  [username, hashed, nombre, rol]);
// Al iniciar sesión: comparar hash
bcrypt.compareSync(password, user.password);
```

¿Qué hace? Nunca almacena contraseñas en texto plano. bcrypt con 10 rondas de sal hace que cada verificación tome ~100ms. Si la BD se ve comprometida, las contraseñas no pueden recuperarse.



Validación + Límite de tamaño CÓDIGO REAL

backend/server.js - Protección contra DoS e inyección de datos

```
// Límite de tamaño del body: 1MB máximo
app.use(express.json({ limit: '1mb' }));
// Validación estricta de tipos
if (typeof username !== 'string' || typeof password !== 'string')
  return res.status(400).json({error:'Datos inválidos'});
// Validación de rol contra lista blanca
if (!['admin','almacen','ventas'].includes(rol))
  return res.status(400).json({error:'Rol inválido'});
```

¿Qué hace? Tres capas: 1) Límite de 1MB para evitar DoS, 2) Validación de tipos (typeof) para evitar inyección de objetos, 3) Lista blanca de roles permitidos.



Husky + lint-staged CÓDIGO REAL

.husky/pre-commit y package.json - Calidad antes de cada commit

```
# .husky/pre-commit - Se ejecuta antes de cada "git commit"
npx lint-staged
npx gitleaks detect --source . --no-git --verbose
```

```
# package.json - lint-staged
"lint-staged": {
  "*.ts,js,json,md,css,html": "prettier --write",
  "*.ts,js": "eslint --fix --quiet"
}
```

¿Qué hace? Husky ejecuta hooks antes de cada commit: 1) lint-staged formatea código con Prettier y corrige errores con ESLint. 2) Gitleaks escanea si hay contraseñas o tokens en el código. Si encuentra algo, bloquea el commit.

Gitleaks - Detector de Secretos CÓDIGO REAL

`.husky/pre-commit` - Previene fugas de credenciales

```
# Gitleaks busca patrones como:
# - AWS Access Key ID (AKIA...)
# - GitHub Tokens (ghp_...)
# - Claves privadas RSA
# - URLs de conexión a BD
npx gitleaks detect --source . --no-git --verbose
# Si encuentra un secreto → ERROR → commit bloqueado
# Si no encuentra nada → OK → commit procede
```

¿Por qué es crítico? Un commit con una clave AWS puede comprometer toda la infraestructura. Gitleaks analiza contra 150+ patrones de secretos. Es la primera línea de defensa contra fugas de credenciales en Git.

4. DevSecOps - Escaneos Automáticos + Hardening

Escaneos con Ansible CÓDIGO REAL

`ansible/devsecops/scan.yml` - 5 herramientas de seguridad

```
# 1. npm audit - Vulnerabilidades en dependencias
- command: npm audit --audit-level=high
  register: npm_audit_backend
# 2. Trivy - Escaneo de archivos
- command: trivy fs --severity HIGH,CRITICAL
# 3. Checkov - Escaneo de playbooks Ansible
- command: checkov --directory ./ansible
# 4. SonarQube - Calidad de código
- command: sonar-scanner
# 5. Snyk - Monitoreo continuo
```

```
- command: snyk test --all-projects
```

Herramientas DevSecOps: npm audit (vulnerabilidades npm), Trivy (archivos), Checkov (Ansible), SonarQube (código), Snyk (monitoreo). Todos los resultados se guardan en /var/log/ckj/scans/.

Hardening del Servidor CÓDIGO REAL

ansible/roles/security/tasks/main.yml - Seguridad del sistema

```
# Fail2Ban: bloquea IPs tras 3 intentos SSH
- apt: name=fail2ban state=present
# UFW: firewall - denegar todo, permitir solo necesario
- ufw: rule=allow port=22 proto=tcp # SSH
- ufw: rule=allow port=80 proto=tcp # HTTP
- ufw: rule=allow port=443 proto=tcp # HTTPS
- ufw: rule=deny port=3000 proto=tcp # API
- ufw: state=enabled policy=deny
# Hardening SSH
- lineinfile: line='PermitRootLogin no'
- lineinfile: line='PasswordAuthentication no'
- lineinfile: line='MaxAuthTries 3'
# Actualizaciones automáticas
- apt: name=unattended-upgrades state=present
```

¿Qué hace? Fail2Ban monitorea SSH y bloquea IPs maliciosas. UFW solo permite puertos 22, 80 y 443. SSH se protege sin root, sin contraseñas y con 3 intentos máx. Actualizaciones de seguridad automáticas.

5. Seguridad - Propuesta de Mejora para CKJ

Herramientas que NO están implementadas pero que mejorarían la seguridad del proyecto

Terraform + AWS WAF PROPUESTA

Infraestructura como código con Web Application Firewall en AWS

```
# terraform/waf.tf - ESTO NO ESTÁ EN EL PROYECTO ACTUAL
resource "aws_wafv2_web_acl" "ckj_waf" {
  name     = "ckj-waf"
  scope    = "REGIONAL"
  rule { name = "rate-limit"; priority = 1
    action { block {} }
  }
}
```

```

statement { rate_based_statement {
  limit = 2000; aggregate_key_type = "IP"
}}
}
rule { name = "xss-block"; priority = 2
  action { block {} }
  statement {
    xss_match_statement {
      field_to_match { query_string {} }
    }
  }
}
}
}

```

Cambios necesarios en CKJ: 1) Crear archivos .tf, 2) Migrar BD de SQLite a RDS PostgreSQL (cambiar sql.js por pg), 3) Subir imagen Docker a ECR, 4) Configurar WAF + CloudFront, 5) Mover uploads/ a S3. **Tiempo estimado:** 2-3 semanas.

Kubernetes Secrets + Vault PROPUESTA

Gestión centralizada y rotación automática de secretos

```

# k8s/secrets.yaml - ESTO NO ESTÁ EN EL PROYECTO ACTUAL
apiVersion: v1
kind: Secret
metadata:
  name: ckj-secrets
type: Opaque
stringData:
  JWT_SECRET: "${JWT_SECRET}"
  DB_PASSWORD: "${DB_PASSWORD}"

# backend/server.js - YA SOPORTA variables de entorno
const JWT_SECRET = process.env.JWT_SECRET
|| 'ckj-almacen-secret-2026'; // fallback

```

Cambios necesarios en CKJ: 1) El código ya lee JWT_SECRET de variable de entorno. 2) Solo falta crear los Secrets en K8s y conectarlos al Deployment. 3) HashiCorp Vault para rotación automática cada 30 días. **Tiempo estimado:** 1-2 semanas.



Pilar 2: Resiliencia

La resiliencia es la capacidad del sistema de mantener su funcionamiento ante fallos, picos de tráfico o condiciones adversas. En CKJ hemos implementado múltiples capas de auto-recuperación: desde el proceso (PM2), pasando por el contenedor (Docker), hasta la infraestructura (Ansible, Nginx).

6. Resiliencia Implementada en CKJ

PM2 - Process Manager CÓDIGO REAL

ansible/roles/app/tasks/main.yml - Gestión de procesos con auto-recuperación

```
# PM2: auto-reinicio + límite de memoria
pm2 start {{ backend_dir }}/server.js \
  --name ckj-api \
  --watch                # Reinicia si cambia código
  --ignore-watch="*.sqlite" # No reinicia por BD
  --max-memory-restart 300M # Reinicia si excede 300MB

# Inicio automático al encender servidor
pm2 startup systemd
pm2 save
```



¿Cómo funciona? Si el proceso se cae por error o memory leak, PM2 reinicia en <2s. El modo watch detecta cambios en código. PM2 startup systemd arranca la app al iniciar el servidor. Límite de 300MB evita que un leak consuma toda la RAM.

Docker - Aislamiento CÓDIGO REAL

Dockerfile - Multi-stage build, imagen mínima y segura

```
FROM node:22-alpine AS build-frontend
WORKDIR /app
COPY package*.json ./; RUN npm ci
COPY . .; RUN npm run build -- --prod

FROM node:22-alpine AS build-backend
COPY backend/ ./backend/
RUN cd backend && npm ci

FROM node:22-alpine # Imagen final: solo lo necesario
RUN apk add --no-cache tini
RUN addgroup -S ckj && adduser -S ckj -G ckj
USER ckj # Ejecutar como no-root
EXPOSE 3000
```

```
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "backend/server.js"]

# docker run -d -p 3000:3000 --restart always ckj:latest
```

Beneficio: La imagen final se reduce de ~1.5GB a ~300MB. --restart always reinicia automáticamente. Tini maneja señales SIGTERM. Usuario no-root limita permisos.

7. Resiliencia - Rollback + Nginx

◀ Ansible Rollback CÓDIGO REAL

ansible/rollback.yml - Recuperación ante fallos en <2 minutos

```
# Volver a versión anterior
ansible-playbook rollback.yml -e "git_ref=v1.0.0"

# El playbook ejecuta automáticamente:
# 1. Git checkout a versión estable
# 2. Reinstala dependencias (npm ci)
# 3. Recompila Angular (ng build --prod)
# 4. pm2 restart ckj-api
# Tiempo total: < 2 minutos
```

☒ Bug en prod ↓ ↓ ☒ Versión estable

🌐 Nginx - Caché + Seguridad CÓDIGO REAL

ansible/roles/nginx/templates/ckj.nginx.conf.j2

```
server {
    listen 443 ssl http2;
    server_name ckj.example.com;

    # SSL
    ssl_certificate      {{ ssl_cert_path }};
    ssl_certificate_key  {{ ssl_key_path }};

    # Frontend Angular (caché 1 año)
    location / {
        root {{ frontend_dir }};
        try_files $uri $uri/ /index.html;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # API Backend (proxy)
```



```
location /api/ {
    proxy_pass http://127.0.0.1:3000;
}

# Seguridad: bloquear rutas sensibles
location ~ /\. { deny all; }
location ~ /(node_modules|\.git|src) { deny all; }
```

¿Qué hace? Nginx actúa como reverse proxy: el frontend se cachea por 1 año (los usuarios siguen viendo la página aunque el backend tenga problemas). SSL/HTTPS cifra la comunicación. Bloquea acceso a rutas sensibles (.git, node_modules).

8. Resiliencia - Propuesta de Mejora



Kubernetes Deployment + HPA PROPUESTA

Orquestación de contenedores con auto-recuperación y escalado horizontal

```
# k8s/deployment.yaml - NO ESTÁ EN EL PROYECTO
apiVersion: apps/v1
kind: Deployment
metadata: { name: ckj-api }
spec:
  replicas: 3 # Múltiples instancias
  selector: { matchLabels: { app: ckj } }
  template:
    metadata: { labels: { app: ckj } }
    spec:
      containers:
        - name: ckj-api
          image: ckj:latest
          ports: [{ containerPort: 3000 }]
          resources:
            limits: { memory: "256Mi", cpu: "500m" }
            livenessProbe: # ¿Está vivo?
              httpGet: { path: /materiales, port: 3000 }
            readinessProbe: # ¿Está listo?
              httpGet: { path: /dashboard, port: 3000 }

# HPA: Auto-escalado
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: ckj-api-hpa }
spec:
```

```
minReplicas: 3; maxReplicas: 10
metrics:
- type: Resource
  resource:
    name: cpu
    target: { type: Utilization, averageUtilization: 60 }
```

Cambios en CKJ: 1) Ya dockerizado, crear manifiestos YAML. 2) Migrar BD a PostgreSQL (RDS). 3) Uploads a S3. **Beneficio:** Si un pod falla, K8s lo recrea. Si CPU > 60%, HPA escala. **Tiempo:** 3-4 semanas.

AWS Auto Scaling + ALB PROPUESTA

Escalado automático de instancias EC2 según demanda

```
# terraform/autoscaling.tf - NO ESTÁ EN EL PROYECTO
resource "aws_launch_template" "ckj" {
  image_id = data.aws_ami.amazon_linux_2.id
  instance_type = "t3.medium"
  user_data = base64encode("#!/bin/bash
    docker run -d -p 3000:3000 --restart always ckj:latest")
}
resource "aws_autoscaling_group" "ckj_asg" {
  min_size = 2; max_size = 10; desired_capacity = 2
  launch_template { id = aws_launch_template.ckj.id }
  target_group_arns = [aws_lb_target_group.ckj.arn]
}
resource "aws_autoscaling_policy" "ckj_cpu" {
  policy_type = "TargetTrackingScaling"
  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
    target_value = 60.0 # Escala si CPU > 60%
  }
}
```

Cambios en CKJ: 1) Migrar a RDS PostgreSQL. 2) Crear AMI con Docker. 3) ALB para distribuir tráfico. 4) Uploads a S3. **Escenario:** Black Friday, CPU > 60% → Auto Scaling lanza instancias (2→10). **Tiempo:** 4-6 semanas.



Pilar 3: Contingencia & Disaster Recovery

La contingencia es el conjunto de planes para recuperar el sistema ante un desastre: pérdida de datos, caída del servidor, ataque cibernético o error humano. Un buen plan minimiza el RTO (tiempo de recuperación) y el RPO (pérdida de datos).

9. Contingencia Implementada en CKJ



Persistencia Inmediata CÓDIGO REAL

backend/server.js - Backup continuo en cada operación

```
function saveDb() {
  // Exporta TODA la BD a disco
  const data = db.export();
  fs.writeFileSync(DB_PATH, Buffer.from(data));
}

function runSql(sql, params = []) {
  db.run(sql, params);
  saveDb(); // Persistencia en CADA operación
}
```

¿Qué hace? Cada INSERT, UPDATE o DELETE guarda la BD al instante. Si el servidor se apaga, solo se pierde la operación en curso. Es un backup continuo. database.sqlite puede respaldarse externamente con un simple copy.



Rate Limiting (Anti-DoS) CÓDIGO REAL

backend/server.js - Protección contra denegación de servicio

```
// 100 peticiones cada 15 minutos por IP
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 100,
  message: { error: 'Demasiadas solicitudes' }
});

app.use('/api/', limiter);

// Solo 5 intentos de login cada 15 min
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 5,
```

```
message: { error: 'Demasiados intentos' }  
});  
app.post('/api/login', loginLimiter, ...);
```

¿Qué hace? Si un atacante intenta saturar la API, responde con 429 Too Many Requests. El login solo permite 5 intentos cada 15 minutos, bloqueando ataques de fuerza bruta a contraseñas.

10. Contingencia - Propuesta de Mejora

RDS Multi-AZ + Backup PROPUESTA

Base de datos redundante en múltiples zonas de disponibilidad

```
# terraform/rds.tf - NO ESTÁ EN EL PROYECTO ACTUAL  
resource "aws_db_instance" "ckj_db" {  
  engine          = "postgres"  
  instance_class = "db.t3.medium"  
  multi_az        = true  # Réplica en otra zona  
  backup_retention_period = 30    # 30 días de backups  
  backup_window    = "03:00-04:00"  
  deletion_protection = true  
}
```

```
# Cambios necesarios en CKJ:  
# 1. Reemplazar sql.js por pg (node-postgres)  
# 2. Las consultas SQL actuales son compatibles  
# 3. Agregar pool de conexiones (pg-pool)  
# 4. Configurar vía variables de entorno
```

Beneficio: Si la BD primaria falla, RDS hace failover a la réplica Multi-AZ en <60 segundos, sin pérdida de datos. Backups automáticos permiten restaurar a cualquier punto en los últimos 30 días. **Tiempo:** 2-3 semanas.

K8s Disaster Recovery Multi-Zona PROPUESTA

Plan de recuperación ante desastres con Kubernetes multi-zona

```
# k8s/pod-disruption-budget.yaml  
apiVersion: policy/v1  
kind: PodDisruptionBudget  
metadata: { name: ckj-pdb }
```

```
spec:
  minAvailable: 2 # Mínimo 2 pods activos
  selector: { matchLabels: { app: ckj } }

# Distribuir pods entre zonas
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule

# Cambios necesarios en CKJ:
# 1. EKS multi-zona (us-east-1a, 1b, 1c)
# 2. BD: RDS Multi-AZ (no SQLite local)
# 3. Uploads: S3 (no disco local)
# 4. Frontend: CloudFront + S3
```

Beneficio: PodDisruptionBudget garantiza ≥ 2 pods activos. TopologySpreadConstraints distribuye pods entre zonas. Si una zona completa falla, las otras siguen sirviendo. **Tiempo:** 4-5 semanas.

11. Tabla Comparativa Completa

20 herramientas implementadas en CKJ + 8 propuestas de mejora

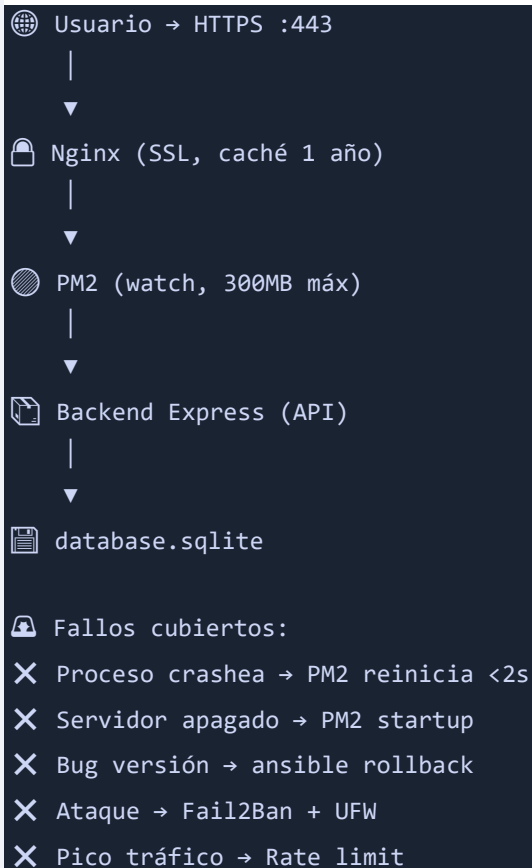
Herramienta	Propósito	Estado	Cambio necesario
Helmet	Cabeceras HTTP seguras	✓ CÓDIGO	—
Rate Limiting	100 req/15min, 5 login	✓ CÓDIGO	—
JWT Auth	Tokens 12h, RBAC	✓ CÓDIGO	—
bcrypt	Hash 10 rondas sal	✓ CÓDIGO	—
Validación	Tipos, roles, 1MB	✓ CÓDIGO	—
Husky	Pre-commit hooks	✓ CÓDIGO	—

lint-staged	Prettier + ESLint	✓ CÓDIGO	—
Gitleaks	Detección de secretos	✓ CÓDIGO	—
npm audit	Vulnerabilidades npm	✓ ANSIBLE	—
Trivy	Escaneo archivos	✓ ANSIBLE	—
SonarQube	Calidad código	✓ ANSIBLE	—
Snyk	Monitoreo vulnerabilidades	✓ ANSIBLE	—
Checkov	Escaneo Ansible	✓ ANSIBLE	—
PM2	Auto-reinicio + 300MB	✓ ANSIBLE	—
Docker	Multi-stage build	✓ DOCKER	—
Ansible	Deploy + rollback	✓ ANSIBLE	—
Nginx	SSL + caché + proxy	✓ ANSIBLE	—
Fail2Ban	Bloqueo SSH	✓ ANSIBLE	—
UFW	Firewall 22,80,443	✓ ANSIBLE	—
saveDb()	Persistencia inmediata	✓ CÓDIGO	—
Terraform	IaC en AWS	🔗 PROP.	Crear .tf + VPC + RDS
AWS WAF	Firewall web OWASP	🔗 PROP.	ACL + CloudFront
Kubernetes	Orquestación + HPA	🔗 PROP.	Manifiestos YAML + EKS
AWS Auto Scaling	Escalado CPU > 60%	🔗 PROP.	Launch template + ASG

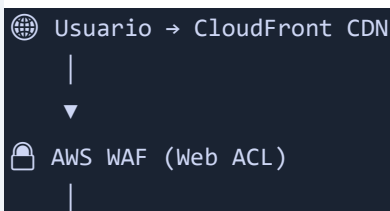
RDS Multi-AZ	BD redundante	 PROP.	sql.js → pg
S3	Uploads escalables	 PROP.	multer-s3
CloudFront	CDN frontend	 PROP.	Build → S3 → CF
HashiCorp Vault	Rotación secretos	 PROP.	Vault Agent

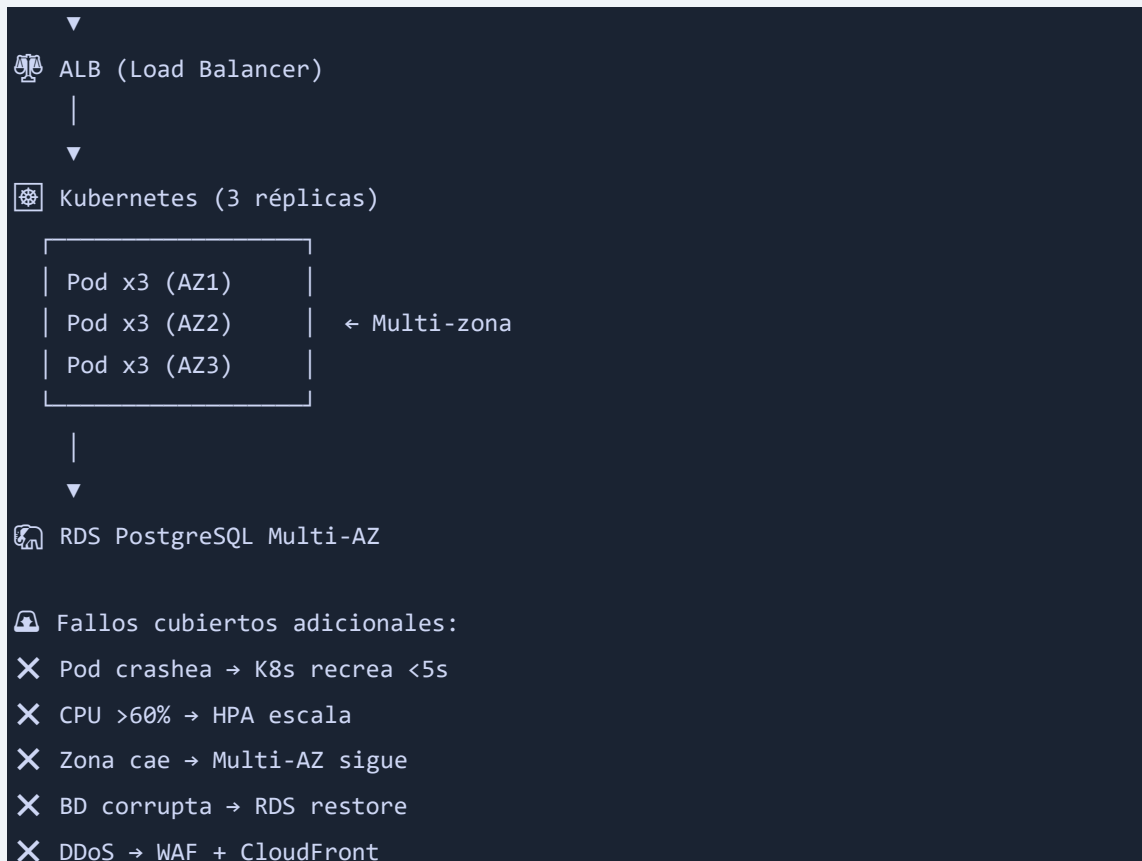
12. Diagrama de Arquitectura

Actual IMPLEMENTADO



Propuesto MEJORA





Conclusión



20

medidas implementadas



8

propuestas de mejora



5

herramientas DevSecOps



Husky

+ Gitleaks

Hoja de ruta propuesta

Fase	Tiempo	Qué
1	1-2 sem	Terraform + S3
2	2-3 sem	RDS PostgreSQL
3	3-4 sem	Kubernetes + HPA
4	4-6 sem	AWS Auto Scaling
5	6-8 sem	Disaster Recovery

La combinación de buenas prácticas, automatización e infraestructura robusta permite construir sistemas resilientes y altamente disponibles.

¡Gracias!

CKJ - Sistema de Almacén

Seguridad, Resiliencia y Contingencia en DevOps

CKJ - Sistema de Almacén © 2026